

Autonomous Multi-Agent Architecture

Orchestration, Swarm Topologies & Memory Systems

Executive Summary: This document provides actionable production implementations engineered by Pinnacle AI Solutions. Use this blueprint to architect zero-downtime, autonomous AI agent pipelines that streamline operational scale and client acquisition.

1. Orchestration Topologies: Peer vs. Hierarchical Swarms

Complex software development cannot rely on a single monolithic LLM prompt. Production systems require multi-agent division of labor: a Master Orchestrator, a Code Implementer, an Adversarial Reviewer, and a QA Verification Subagent.

In hierarchical topologies, the orchestrator decomposes requirements into discrete, atomic subtasks, spawning parallel CLI subagents that report findings through shared memory.

- Master Orchestrator: Maintains project context, allocates compute budgets, and tracks progress.
- Domain Specialists: Dedicated agents for database migrations, frontend UI tokens, and API endpoints.
- Adversarial Reviewer: Red-team checks for edge cases, resource leaks, and security vulnerabilities.

2. Context Preservation & Token Window Budgeting

Agent degradation is predominantly caused by context poisoning, lost-in-the-middle phenomena, and token bloating. High-performance workflows implement recursive compaction and structured memory banks.

- Task Boundaries: Explicit state checkpoints before and after major file modifications.
- Semantic Compression: Strip voluminous build logs before passing summaries to child agents.
- Dynamic Tool Gating: Lazy-load MCP schemas only when required by the active plan step.

```
// Standardized Agent Memory State Contract
{
  "session_id": "pinnacle-prod-v2",
  "orchestrator_state": "ACTIVE",
  "active_phase": "DEPLOY_AUTHENTICATION",
  "completed_milestones": ["SCHEMA_MIGRATION", "UI_TOKEN_AUDIT"],
  "context_preservation_score": 0.985
}
```

3. Failure Recovery & Self-Healing Loops

When a subagent fails or encounters an unexpected exception, the orchestrator triggers an automatic postmortem cycle, capturing the stdout/stderr trace, restoring the last known good git stash, and generating a corrective prompt.